Wi-Fi Intrusion Prevention System

A machine-learning prototype that detects deauthentication flood attacks and automatically blocks the offending MAC address.

Author: Chiwo Newa
Student ID: 190358
Module: CNS 320;
Date: 26 Oct 2026

1. The Problem

A **deauthentication flood** is a trivial denial-of-service attack on 802.11: an attacker spoofs management frames to force every client off the network. It needs no password and no special hardware.

This project builds a transparent end-to-end prototype: *capture → features → ML → alert → block*, using a **Random Forest** classifier.

2. System Architecture

Packet source
synthetic · AWID3 CSV · monitor NIC

↓

Windowing
1-second buckets per (src MAC)

↓

Feature extraction
11 numeric features

↓

Random Forest
100 trees · depth 12 · balanced

↓

Blocklist
MAC added after 2 strikes

3. Features (why these 11?)

Each window becomes one row. Ratios (not just counts) make the model *shape-aware* — a busy legitimate AP sends hundreds of frames/s too.

Feature	Catches
`deauth_count`	the flood
`deauth_ratio`	burst *shape*
`frame_count`	total rate
`mgmt_ratio`	data-starved flood
`rssi_std`	uniform-power burst
`unique_dst`	broadcast targeting

4. Results

Evaluated on two independent data sources. Both achieved perfect detection, proving the pipeline generalises beyond synthetic quirks.

Synthetic Lab Traffic

100% ACCURACY — **100%** RECALL

	pred normal	pred attack
actual normal	2,870	0
actual attack	0	32

AWID3 Real-World Dataset

100% ACCURACY — **100%** RECALL

	pred normal	pred attack
actual normal	326	0
actual attack	0	89

AWID3 run used 65,488 frames (20.8% attack) from the 1.Deauth subset.

5. The "P" in WIPS

When a window is classified *attack*, the detector raises an alert and increments a **strike counter** for that MAC. After **2 strikes**, the MAC is written to a persistent blocklist.

```
t=3s normal traffic
t=47s ⚠ ALERT aa:bb:..:02 p=0.98
t=48s ⚠ ALERT aa:bb:..:02 p=0.97
t=48s 🛡 BLOCK aa:bb:..:02
```

Why two strikes? One alert could be a false positive; two consecutive attacking windows from the same MAC is a very strong signal.

Safety: `DRY_RUN=True` by default. On an authorised lab host, `_apply_block()` runs `iptables -I INPUT -m mac --mac-source <mac> -j DROP`.

6. Reproducibility

- Random Forest, 100 trees, max depth 12, `class_weight="balanced"`.
- Time-ordered 70/30 split — no attack episode appears in both halves.
- Fixed seeds (`RANDOM_STATE=42`).
- Tests: `python tests/test_features.py`.
- Dataset-agnostic: same pipeline runs on AWID3 or synthetic traffic.
- **Dataset:** AWID3-CSV (Kaggle) / Official AWID3.

7. Limitations & Future Work

Limitations

- Strong signal in `deauth_count`; other attacks need re-training.
- Each window is scored independently.
- Offline — reads CSVs; live capture needs a monitor-mode NIC.

Would improve

- Rolling 5 s features to smooth isolated false positives.
- `CalibratedClassifierCV` for trustworthy probabilities.
- Adversarial validation on disassoc / rogue-AP.
- Active deauth back to the attacker (injection).
- Streaming learner for concept drift.

8. AI Assistant Use

> **Declaration.** An AI coding assistant (DeepSeek) was used for: initial project scaffolding, drafting the synthetic traffic simulator, first-pass hyperparameter suggestions, and drafting docstrings. All output was reviewed and tested by the author — notably the time-ordered split and the two-strike blocking policy replaced the assistant's weaker suggestions. The author can explain every line during the live demo.

9. Live Demo

1. `python -m src.run_demo` — trains on 10 min synthetic, replays a fresh attack.
2. Watch the alert timeline and blocklist grow.
3. `python -m src.run_awid3 --data-dir …` — same pipeline, real dataset.

All traffic is synthetic or public AWID3. No live radio transmission by this code.